

Resumo do Artigo: Design by Contract (Projeto por Contrato)

Autor: Bertrand Meyer, Interactive Software Engineering Inc.

Aluno: Nicolas Araújo Fonseca Pimenta

1. Introdução e Objetivo do Artigo

O artigo apresenta uma proposta de Bertrand Meyer para tornar o software mais confiável, ou seja, ao mesmo tempo correto e robusto. O autor parte de uma constatação: muitas pesquisas teóricas sobre confiabilidade de software, como verificação formal e especificação, nunca chegaram à prática do dia a dia dos desenvolvedores. Pior ainda, boa parte da literatura sobre programação orientada a objetos foca em flexibilidade e reuso, mas fala pouco sobre como garantir que o código realmente faça o que promete. Para resolver isso, Meyer propõe enxergar a construção de software como uma série de decisões de contratação documentada, ou seja, cada parte do programa funciona como um contrato entre duas partes. Essa ideia é a base do que ficou conhecido como Design by Contract, ou Projeto por Contrato, e foi implementada principalmente na linguagem Eiffel, criada pelo próprio autor.

2. A Noção de Contrato no Software

Meyer compara o desenvolvimento de software com a vida real. Quando alguém contrata um serviço, por exemplo a construção de um prédio, existe um documento que define direitos e deveres dos dois lados. O cliente precisa fornecer o terreno, e o construtor precisa entregar o prédio dentro das condições combinadas. Esse contrato protege os dois: o cliente sabe o que vai receber, e o construtor sabe até onde vai sua responsabilidade. No software acontece a mesma coisa quando uma rotina chama outra rotina. O autor defende que toda chamada de rotina deveria ter um contrato claro, com obrigações e benefícios para o cliente, que é quem chama, e para o fornecedor, que é quem executa. Esse contrato traz quatro pontos importantes:

- **Obrigações mútuas:** a obrigação de um lado é o benefício do outro. Se o cliente faz sua parte, o fornecedor precisa cumprir o que prometeu.
- **Explicitação dos termos:** o contrato precisa estar escrito de forma clara, sem ambiguidades, para que qualquer pessoa possa entender o que está combinado.
- **Cláusulas gerais implícitas:** existem regras que valem para todos os contratos, como leis da sociedade, e que não precisam ser repetidas em cada um.
- **Regra das cláusulas não ocultas:** uma lista de obrigações também funciona como um benefício, pois limita o que pode ser exigido daquela parte. Ninguém pode cobrar mais do que está no contrato.

3. Asserções: Pré-condições, Pós-condições e Invariantes

Para colocar o contrato no código, Meyer apresenta o conceito de asserções, que são expressões booleanas usadas para definir as regras dos contratos. Existem três tipos principais:

- **Pré-condições:** ficam no início de uma rotina, marcadas pela palavra-chave **require**. Elas definem o que o cliente precisa garantir antes de chamar a rotina. Por exemplo, antes de inserir um elemento em uma tabela cheia, é obrigatório verificar se ainda existe espaço disponível.
- **Pós-condições:** ficam no final da rotina, marcadas pela palavra-chave **ensure**. Elas definem o que a rotina promete entregar ao terminar, desde que a pré-condição tenha sido cumprida. Por exemplo, depois da inserção, a tabela vai ter um elemento a mais e esse elemento estará associado à chave informada.
- **Invariantes de classe:** são regras que valem para todos os objetos de uma classe e devem ser sempre verdadeiras, tanto depois da criação do objeto quanto depois de qualquer chamada de método. Por exemplo, em uma tabela, a quantidade de elementos sempre deve estar entre zero e a capacidade máxima.

Meyer destaca que essas asserções não são apenas verificações, e sim documentação viva do software. Elas mostram, de forma precisa, qual é a intenção de cada rotina, sem precisar olhar a implementação por dentro.

4. Contra a Programação Defensiva

Um dos pontos mais polêmicos e interessantes do artigo é a crítica à programação defensiva. A programação defensiva é a prática tradicional de proteger cada rotina com várias verificações, mesmo quando essas verificações já foram feitas por quem chamou a rotina. Meyer argumenta que essa prática, em vez de ajudar, atrapalha, pois aumenta a complexidade do código e, ironicamente, reduz a confiabilidade. Quando todo mundo verifica tudo, surgem redundâncias, duplicações e até inconsistências entre as verificações. A proposta do Design by Contract é diferente: define-se de forma clara quem é responsável por cada condição. Ou a condição faz parte da pré-condição, e o cliente é responsável, ou a condição não faz parte da pré-condição, e o fornecedor lida com ela. Nunca os dois ao mesmo tempo. Isso elimina verificações redundantes e deixa o código mais simples e claro.

5. Tratamento de Exceções e Situações Anormais

Meyer dedica uma grande parte do artigo a discutir como lidar com situações anormais. Ele define que uma situação anormal acontece quando uma das partes do contrato não consegue cumprir sua obrigação. O autor critica fortemente os mecanismos de exceção de linguagens como Ada e CLU, mostrando exemplos em que o uso de exceções esconde erros graves do programa, como uma rotina de raiz quadrada que retorna como se nada tivesse acontecido quando recebe um número negativo. Para resolver isso, ele propõe três leis do tratamento de exceções:

- **Primeira lei:** uma chamada de rotina só pode terminar de duas formas, ou ela cumpre o contrato, ou ela falha em cumprir. Não existe meio termo, nem fingir que deu tudo certo quando na verdade não deu.
- **Segunda lei:** a falha de uma rotina sempre deve gerar uma exceção na rotina que a chamou. Ou seja, a rotina que falhou precisa avisar quem chamou, e não pode esconder o problema.

- **Terceira lei:** diante de uma exceção, só existem duas respostas válidas. A primeira é a retomada, em que a rotina tenta novamente, com a mesma estratégia ou com outra. A segunda é o pânico organizado, em que a rotina coloca os objetos em um estado estável e avisa o chamador que falhou.

Para implementar isso, Eiffel oferece dois mecanismos simples: a cláusula **rescue**, que define o comportamento alternativo quando algo dá errado, e a instrução **retry**, que permite tentar executar a rotina de novo. Esse modelo é considerado pelo autor mais simples e seguro do que os modelos de outras linguagens.

6. Herança e Vinculação Dinâmica como Subcontratação

Outro ponto fundamental do artigo é o uso da teoria dos contratos para explicar herança e vinculação dinâmica na programação orientada a objetos. Meyer defende que herança com redefinição funciona como subcontratação. Quando uma classe filha redefine um método de uma classe pai, é como se um contratante terceirizasse uma tarefa para um subcontratado. Mas, assim como na vida real, o subcontratado precisa respeitar o contrato original. A partir disso, Meyer enuncia a regra de redefinição:

- **Pré-condição mais fraca:** a versão redefinida do método pode aceitar mais casos, ou seja, ser menos exigente com quem chama. Mas nunca pode ser mais exigente do que o método original.
- **Pós-condição mais forte:** a versão redefinida pode entregar resultados melhores ou mais detalhados do que o método original. Mas nunca pode entregar menos do que o prometido.

Em Eiffel, essas regras são garantidas pela linguagem por meio das palavras-chave **require else** e **ensure then**. O autor também faz uma crítica severa ao uso de vinculação estática em linguagens que se dizem orientadas a objetos. Para ele, a vinculação estática é um erro grave, pois pode levar à execução de uma versão errada do método e produzir objetos em estados inconsistentes, o que é o pior cenário possível.

7. Defesa das Funções Parciais

No fim do artigo, Meyer faz uma defesa explícita do uso de funções parciais, que são funções que não são definidas para todos os valores possíveis de entrada. Um exemplo clássico é a função que calcula o inverso de um número, que não é definida para zero. Muitos autores e linguagens evitam funções parciais porque acreditam que elas tornam o software menos robusto. Meyer discorda dessa visão e argumenta que a robustez de um sistema não vem da tolerância exagerada de cada rotina, e sim da coerência geral da arquitetura, da consistência das interfaces entre módulos e da simplicidade de cada parte. Para ele, quando os contratos estão bem definidos por meio de pré-condições e pós-condições, é melhor deixar cada rotina fazer bem um trabalho específico do que tentar cobrir todos os casos possíveis. O excesso de verificações leva a interfaces complicadas e a um software menos confiável, ao contrário do que se costuma pensar.

8. Conclusão

O artigo de Bertrand Meyer apresenta uma visão completa e bem fundamentada sobre como construir software confiável. A ideia central, a metáfora do contrato, oferece um

modelo simples e poderoso para pensar em rotinas, classes, herança e tratamento de erros. O Design by Contract trouxe contribuições importantes para a engenharia de software, como o uso de asserções para documentar e validar código, a crítica à programação defensiva, um modelo mais coerente de tratamento de exceções e uma forma mais clara de entender herança como subcontratação. Mesmo sendo um artigo de 1991, baseado na linguagem Eiffel, as ideias apresentadas continuam relevantes até hoje e influenciaram muitas práticas modernas de programação. O autor conclui que aceitar funções parciais como algo natural, deixar cada componente do sistema dizer claramente o que faz e o que não faz, e definir contratos precisos entre as partes, é o caminho mais realista para construir software de qualidade.